

Session 01

Generative AI for Data Work

หลักสูตร: การใช้ Generative AI เพื่อการวิเคราะห์ข้อมูลสำหรับภาครัฐ

Asst. Prof. Taweesak Samanchuen, Ph.D.

Mahidol University

วัตถุประสงค์การเรียนรู้

- เข้าใจความหมายของ **Generative AI** และ **LLM**
- เข้าใจหลักการทำงานของ **Prompt, Token** และ **Context Window**
- เข้าใจหลักการ **Prompt Engineering** และเทคนิคการปรับ Prompt ให้ได้ผล
- สามารถปรับ Prompt ให้เหมาะกับงานข้อมูลที่ทำอยู่จริง

01

Generative AI and LLM

Using AI to Accelerate Data Work, Not Replace Thinking

AI คืออะไร? (ฉบับ 2 นาที)

AI (Artificial Intelligence)

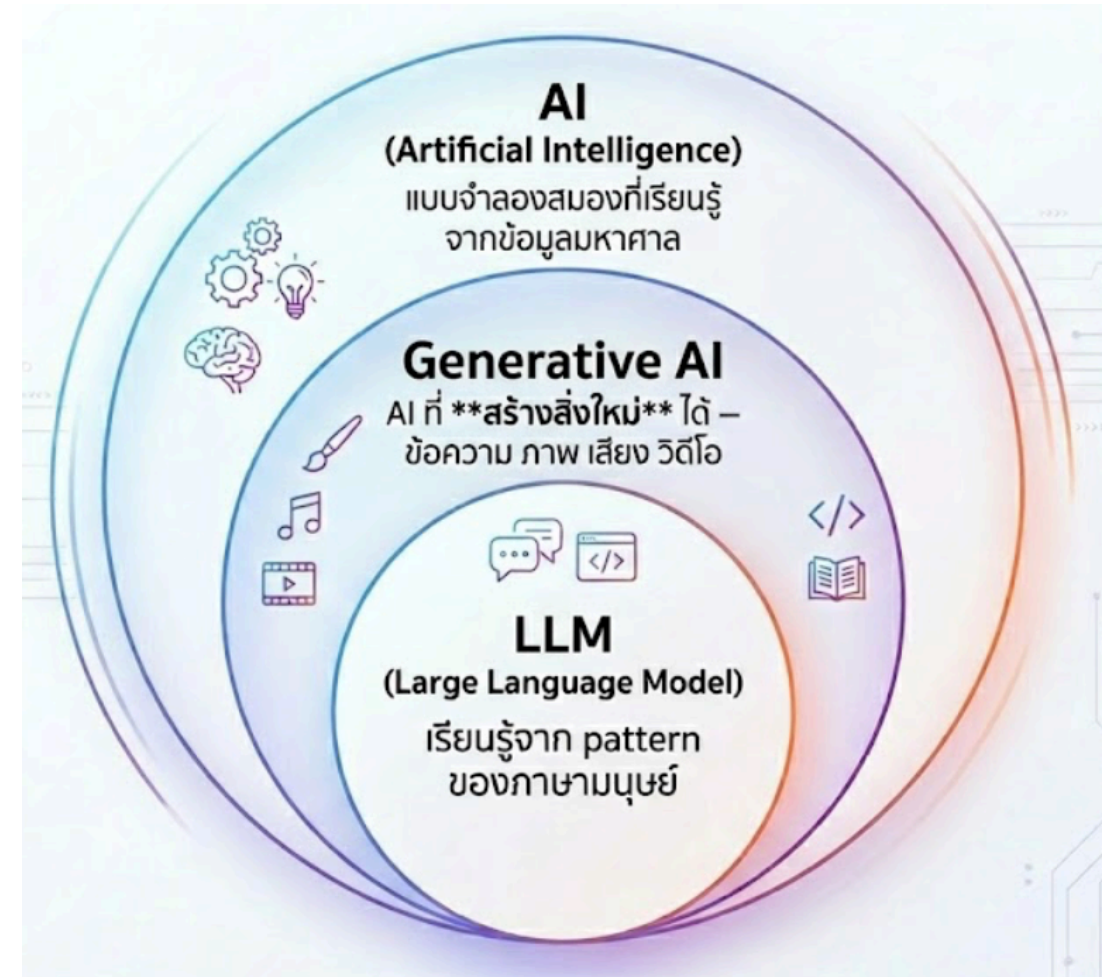
แบบจำลองสมองที่เรียนรู้จากข้อมูลมหาศาล

Generative AI

AI ที่ **สร้างสิ่งใหม่** ได้ — ข้อความ ภาพ เสียง วิดีโอ

LLM (Large Language Model)

เรียนรู้จาก pattern ของภาษามนุษย์



Generative AI คืออะไร

คำอธิบายแบบสั้นที่สุด

- เป็นกลุ่มของ AI ที่ **สร้างเนื้อหาใหม่** จาก pattern ที่เรียนรู้มา
- เนื้อหาที่สร้างได้อาจเป็น **ข้อความ, รูปภาพ, เสียง, โค้ด, วิดีโอ**
- คำว่า "generate" ไม่ได้แปลว่าคิดเองจากศูนย์ แต่คือสร้าง output ใหม่จากสิ่งที่เคยเรียนรู้

ตัวอย่างที่พบได้บ่อย

- สร้างอีเมลหรือสรุปรายงาน
- สร้างรูปภาพจาก prompt
- สร้างโค้ด Python หรือ SQL
- สร้างข้อความจำลองสำหรับทดสอบระบบ

ตัวอย่างการใช้งาน Generative AI ในปัจจุบัน

สิ่งที่เห็น	จริง ๆ คืออะไร	ตัวอย่างงาน
ChatGPT, Claude, Gemini	ผู้ช่วยสนทนาที่ขับเคลื่อนด้วย LLM	ถามตอบ, สรุป, เขียน SQL
Llama, GPT, Qwen	ตระกูลของ LLM	generate text, classify, extract
Midjourney, DALL-E	Generative AI สำหรับภาพ	สร้างรูปจากข้อความ
Suno	Generative AI สำหรับเสียง/เพลง	สร้าง audio

บริบทของคาบนี้

- เราจะโฟกัส **text-based Generative AI**
- หรือพูดให้ตรงขึ้นคือ **การใช้ LLM กับงานข้อมูล**

แล้ว LLM คืออะไร

ความสัมพันธ์กับ Generative AI

- **LLM = Large Language Model**
- LLM เป็น **หนึ่งประเภทของ Generative AI** ที่เชี่ยวชาญเรื่องภาษา
- ถ้า Generative AI เป็น "หมวดใหญ่" LLM คือ "เครื่องมือย่อย" ที่ใช้สร้างและเข้าใจข้อความ

เปรียบเทียบแบบง่าย

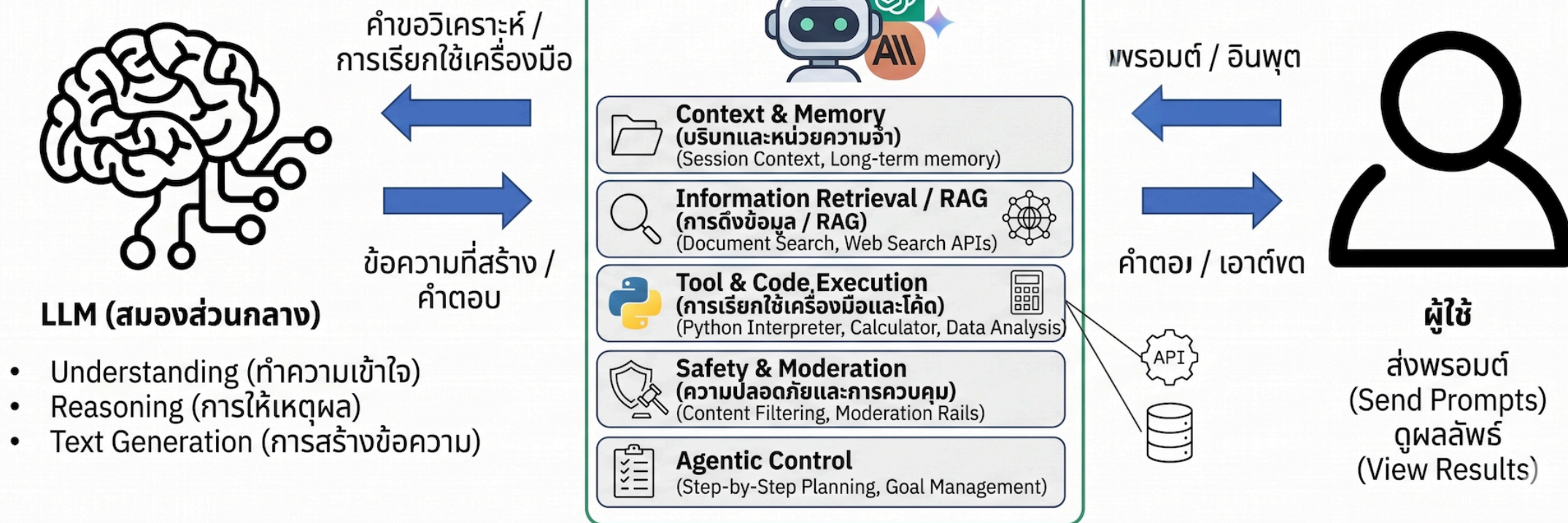
- **Generative AI:** คำรวมของระบบที่สร้าง content ใหม่
- **LLM:** โมเดลที่สร้างและประมวลผล "ภาษา" เป็นหลัก

สรุปสั้น: ทุก LLM เป็น Generative AI แต่ไม่ใช่ทุก Generative AI จะเป็น LLM

Modern Chat AI

สถาปัตยกรรมแบบบอท AI สมัยใหม่ (เช่น ChatGPT, Gemini, Claude)

เลเยอร์ประสานงานแบบบอท



Token คืออะไร?

Token = หน่วยเล็กที่สุดที่ AI ใช้ "อ่าน" และ "เขียน" ข้อความ — ไม่ใช่คำ แต่เป็นชิ้นส่วนของคำ

ตัวอย่าง (ภาษาอังกฤษ)

- "Hello" = 1 token
- "unbelievable" = 3 tokens
- 1 คำ $\approx$ 0.75 token โดยเฉลี่ย

ตัวอย่าง (ภาษาไทย)

- ภาษาไทยใช้ token มากกว่า EN $\sim$ 2-3x
- "สวัสดี" $\approx$ 3-5 tokens

Token คืออะไร? (ต่อ)

ทำไมต้องรู้?

- **Context Window** วัดเป็น token
- ยิ่ง Prompt ยาว → ใช้ token มาก → เหลือพื้นที่คำตอบน้อยลง
- บริการ API คิดเงินตาม token

สำหรับคนทำงานทั่วไป: จำง่าย ๆ ว่า **1,000 token $\approx$ ข้อความ $\frac{3}{4}$ หน้า A4**

สามารถทดสอบนับ token ได้ที่: tiktokenizer.vercel.app

Context Window คืออะไร?

Context Window = "ความจำระยะสั้น" ของ AI — ข้อมูลทั้งหมดที่ AI มองเห็นในการสนทนาครั้งนั้น

ใส่ได้ทั้ง

- ข้อความบทสนทนาทั้งหมด
- เอกสารที่แนบ / วางข้อความ
- คำสั่งระบบ (System Prompt)

Gemini 2.0 รองรับ

- **1 ล้าน token** (~750,000 คำ)
- $\approx$ หนังสือ 10 เล่ม หรือโค้ด 30,000 บรรทัด
- เกิน limit $\rightarrow$ AI "ลืม" ส่วนต้น

💡 ถ้าสนทนาวายาวมากแล้ว AI เริ่มตอบผิดพลาด — ลองเปิด chat ใหม่และสรุป context ก่อน

LLM ช่วยงานข้อมูลอะไรได้บ้าง

ใน workflow จริง

- แปลงคำถามธุรกิจเป็น **SQL**
- ช่วยอ่านผล **EDA** และตั้ง hypothesis
- ดึงข้อมูลจากข้อความด้วย **extraction**
- ตรวจสอบความผิดปกติและช่วย reasoning เบื้องต้น
- สรุปผลเชิงเทคนิคให้เป็นภาษาที่สื่อสารกับทีมได้

คำถามหลักของคาบ: ใช้ AI ตรงไหนแล้วคุ้ม แต่ยังควบคุมคุณภาพได้?

ศัพท์สำคัญที่ต้องรู้ก่อนเริ่ม

คำที่เจอบ่อยในคาบนี้

- **Prompt:** ข้อความคำสั่งที่เราใช้สื่อสารกับโมเดล
- **Context:** ข้อมูลประกอบที่ใส่เพิ่มให้โมเดลตอบดีขึ้น
- **Hallucination:** โมเดลตอบอย่างมั่นใจ แต่ข้อเท็จจริงผิด
- **Human verification:** มนุษย์เป็นผู้ตรวจสอบคำตอบก่อนนำไปใช้จริง
- **Workflow:** ลำดับขั้นตอนของงานตั้งแต่รับข้อมูลจนสื่อสารผล

หลักง่าย

- prompt ที่ดี + context ที่พอ + การตรวจโดยคน = ใช้งานได้จริงมากขึ้น

หลักคิดก่อนใช้ LLM

AI เป็นผู้ช่วย ไม่ใช่ผู้ตัดสินใจสุดท้าย

- LLM เก่งเรื่อง pattern ของภาษา ไม่ได้ "รู้จริง" ทุกเรื่อง
- คำตอบที่สั้นไหลอาจยัง **ผิดเชิงข้อเท็จจริง** ได้
- งาน data ต้องมี **human verification** เสมอ
- ยิง prompt ชัด ยิงลดความคลุมเครือของ output

สิ่งที่ต้องตรวจสอบทุกครั้ง

- logic ถูกไหม
- data leakage หรือไม่
- ตัวเลขและ assumptions สอดคล้องโจทย์หรือไม่

Hallucination คืออะไร

อาการที่เจอได้บ่อย

- อ้างชื่อคอลัมน์ที่ไม่มีอยู่จริง
- เขียน SQL ที่ syntax ดูเหมือนถูก แต่รันไม่ได้
- สรุป insight ที่ไม่ได้รองรับจากข้อมูล
- อ้างเหตุผลเชิงธุรกิจเกินกว่าสิ่งที่ข้อมูลบอกได้

วิธีรับมือ

- ให้ schema หรือข้อมูลต้นทางชัดเจน
- ขอให้โมเดลระบุ assumptions
- ตรวจสอบด้วย code, query, หรือ domain knowledge จริง

ตัวอย่างงานในคาบนี้: อะไรคือ Generative AI

ตัวอย่างที่เราจะเห็นจริง

- ให้ LLM **สร้าง SQL** จากคำถามธุรกิจ
- ให้ LLM **สรุป EDA** จาก `describe()` และ missing values
- ให้ LLM **extract sentiment และ entities** จาก feedback
- ให้ LLM **ร่าง summary ภาษา business** จากผลวิเคราะห์

จุดร่วมของทุกตัวอย่าง

- โมเดลรับ input เป็นข้อความหรือข้อมูลที่แปลงเป็นข้อความ
- โมเดลสร้าง output ใหม่ในรูปแบบที่มนุษย์นำไปใช้ได้
- แต่ผลลัพธ์ยังต้องผ่านการตรวจโดยผู้วิเคราะห์เสมอ

02

Basic Prompt Engineering

Prompt คืออะไร?

Prompt = ประโยคที่คุณพิมพ์ให้ Gemini

ผลลัพธ์ดีหรือแย่ — ขึ้นอยู่กับ **วิธีที่คุณถาม** เป็นหลัก

สูตรพื้นฐาน

บริบท + งาน + รูปแบบที่ต้องการ

"คุณคือผู้ช่วยฝ่าย HR (บริบท) — ช่วยเขียนอีเมลแจ้งนโยบาย WFH ใหม่ให้พนักงาน (งาน) — โทนสุภาพ ความยาว 2 ย่อหน้า (รูปแบบ)"

ก่อน vs. หลัง ใส่ Prompt ที่ดี

Prompt แย่	Prompt ดี
"สรุปรายงานให้หน่อย"	"สรุปรายงานนี้เป็น 5 bullet points เน้น Action Items สำหรับผู้บริหาร"
"เขียนอีเมล"	"เขียนอีเมลทางการถึงลูกค้า เรื่องเลื่อนส่งงาน โทนขอโทษและมั่นใจ"
"แปลให้หน่อย"	"แปลข้อความนี้เป็นภาษาอังกฤษแบบ Business formal"

CO-STAR Framework คืออะไร?

CO-STAR คือกรอบการเขียน Prompt ที่พัฒนาโดย GovTech Singapore เพื่อช่วยให้ได้คำตอบจาก AI ที่ ตรงความต้องการและนำไปใช้งานได้ทันที

ปัญหาที่แก้

- Prompt คลุมเครือ → คำตอบไม่ตรงใจ
- AI เดาเจตนาผิด → ต้องถามซ้ำหลายรอบ
- ผลลัพธ์ไม่เหมาะกับผู้ใช้

💡 ใช้งานได้กับทุก AI — Gemini, ChatGPT, Claude ฯลฯ

วิธีที่แก้

- ใส่ข้อมูล **6 มิติ** ให้ AI เข้าใจงานครบถ้วน
- ลด back-and-forth → ประหยัดเวลา
- ผลลัพธ์พร้อมใช้ตั้งแต่ครั้งแรก

สูตร CO-STAR Framework

ตัวอักษร	ความหมาย	ตัวอย่าง
Context	บริบทของงาน	"ฉันเป็น HR ในบริษัท 200 คน"
Objective	เป้าหมายที่ต้องการ	"ช่วยเขียนอีเมลแจ้งนโยบาย WFH"
Style	รูปแบบการเขียน	"เขียนแบบ Business formal"
Tone	โทนเสียง	"สุภาพ เป็นมิตร ไม่เป็นทางการเกินไป"
Audience	กลุ่มเป้าหมาย	"พนักงานทุกระดับ ไม่เชี่ยวชาญเทคนิค"
Response	รูปแบบผลลัพธ์	"ตอบเป็นอีเมล ความยาว 2 ย่อหน้า"

หลักการใช้ CO-STAR ให้ได้ผล

- **ไม่ต้องครบทุกตัวทุกครั้ง** — งานง่ายใช้แค่ C + O + R ก็พอ
 - **เรียง C → O ก่อนเสมอ** — AI ต้องรู้บริบทและเป้าหมายก่อนตอบ
 - **A (Audience) สำคัญมาก** — "สำหรับผู้บริหาร" vs "สำหรับพนักงานใหม่" ให้ผลต่างกันมาก
 - **T (Tone) ควบคุมน้ำเสียง** — เป็นทางการ / เป็นกันเอง / กระชับตรงประเด็น
- 💡 เริ่มจาก **C + O + R** ก่อน แล้วค่อยเพิ่ม S, T, A เมื่อผลลัพธ์ยังไม่ตรงที่ต้องการ

CO-STAR ในงานจริง — ตัวอย่าง

[C] ฉันเป็นเจ้าหน้าที่ฝ่ายประชาสัมพันธ์ของหน่วยงานราชการ

[O] ช่วยเขียนประกาศรับสมัครงานตำแหน่งนักวิเคราะห์นโยบาย

[S] เขียนแบบราชการ ถูกต้องตามรูปแบบ

[T] เป็นทางการ น่าเชื่อถือ

[A] ผู้สมัครที่จบปริญญาตรีขึ้นไป

[R] ตอบเป็นประกาศพร้อมใช้ ความยาว 1 หน้า A4

💡 ยิ่งให้ข้อมูลครบ — Gemini ยิ่งตอบตรงและนำไปใช้ได้ทันที

เทคนิค: Chain of Thought

ให้ AI "คิดทีละขั้น" ก่อนตอบ

เพิ่มประโยคนี้ในทุก Prompt ที่ต้องการความแม่นยำ:

"ลองคิดทีละขั้นตอน แล้วค่อยตอบ"

หรือ **"Let's think step by step"**

เหมาะกับ: การคำนวณ, การวิเคราะห์เหตุผล, การตัดสินใจซับซ้อน

CoT ปี 2026 — ยังต้องพิมพ์เองไหม?

Model	Reasoning อัตโนมัติ	ต้องสั่งเอง
Gemini 2.0 Flash Thinking	✅ คิดเองโดยไม่ต้องบอก	ไม่จำเป็น
Gemini 1.5 / GPT-4o	⚠️ คิดได้บ้าง	แนะนำสั่ง

💡 ถ้าใช้ **Gemini 2.0+** — เปิด "Thinking Mode" แทนการพิมพ์ "คิดทีละขั้น" ได้เลย

ทำอะไรแทน CoT ได้บ้าง?

- **เปิด Thinking Mode** — Gemini 2.0 Flash Thinking มี reasoning built-in
- **แบ่งถามทีละ step** — ถาม "วิเคราะห์ปัญหาก่อน" → แล้วค่อย "เสนอแนวทาง"
- **ใช้ CO-STAR ให้ครู** — O และ R ที่ชัดเจนช่วยให้ AI reasoning ดีขึ้นเอง

แทนที่: "วิเคราะห์สัญญาณนี้ให้หน่อย"

ใช้: "อ่านสัญญาณนี้ก่อน → แล้วบอกว่าข้อไหนเสี่ยง → พร้อมแนะนำว่าควรทำอะไร"

เทคนิค: Role Prompting

กำหนด "บทบาท" ให้ AI ก่อนถาม

"คุณคือ **นักกฎหมายผู้เชี่ยวชาญด้านสัญญา**

อ่านสัญญาต่อไปนี้และบอกจุดเสี่ยงที่ควรระวัง: [วางสัญญา]"

💡 Role ที่ดีช่วยให้ Gemini ใช้ "มุมมอง" ที่ถูกต้อง — ผลลัพธ์แม่นยำกว่ามาก

Role Prompting — ตัวอย่างในงานออฟฟิศ

Role ที่กำหนด	เหมาะกับงาน
"คุณคือนักสื่อสารองค์กรมืออาชีพ"	เขียนประกาศ อีเมล จดหมาย
"คุณคือวิทยากรที่สอนคนไม่เชี่ยวชาญ"	อธิบายเรื่องยากให้เข้าใจง่าย
"คุณคือบรรณาธิการภาษาไทย"	ตรวจแก้ภาษา ปรับโทนเนื้อหา
"คุณคือผู้จัดการโครงการที่มีประสบการณ์"	วางแผน ระบุความเสี่ยง ติดตามงาน

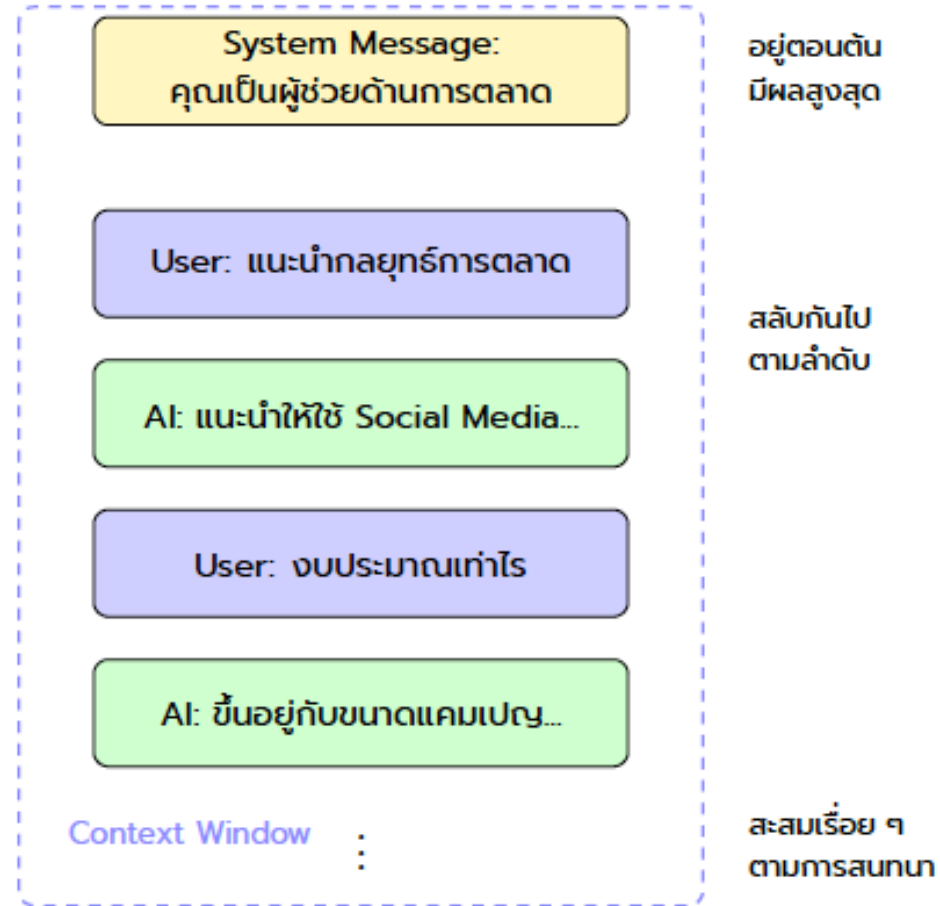
💡 เลือก Role ที่ **ไม่ขึ้นกับกฎหมายหรือข้อบังคับเฉพาะประเทศ** — ผลลัพธ์จะแม่นยำและเชื่อถือได้มากกว่า

Chat Template Structure คืออะไร?

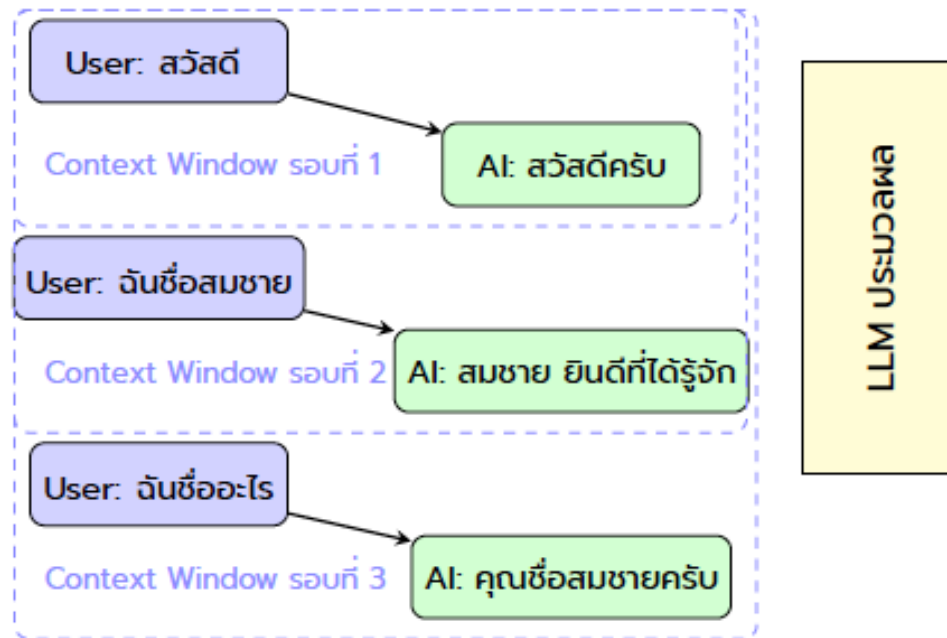
ทุก Prompt ที่คุณส่ง จริงๆ แล้ว AI มองเห็นเป็น 3 ส่วน

ส่วน	คือ	ตัวอย่าง
System	คำสั่งตั้งต้น / บุคลิก AI	"คุณคือผู้ช่วยที่ตอบภาษาไทยเท่านั้น"
User	สิ่งที่คุณพิมพ์	Prompt ที่คุณส่งไป
Assistant	คำตอบของ AI	ข้อความที่ Gemini ตอบกลับ

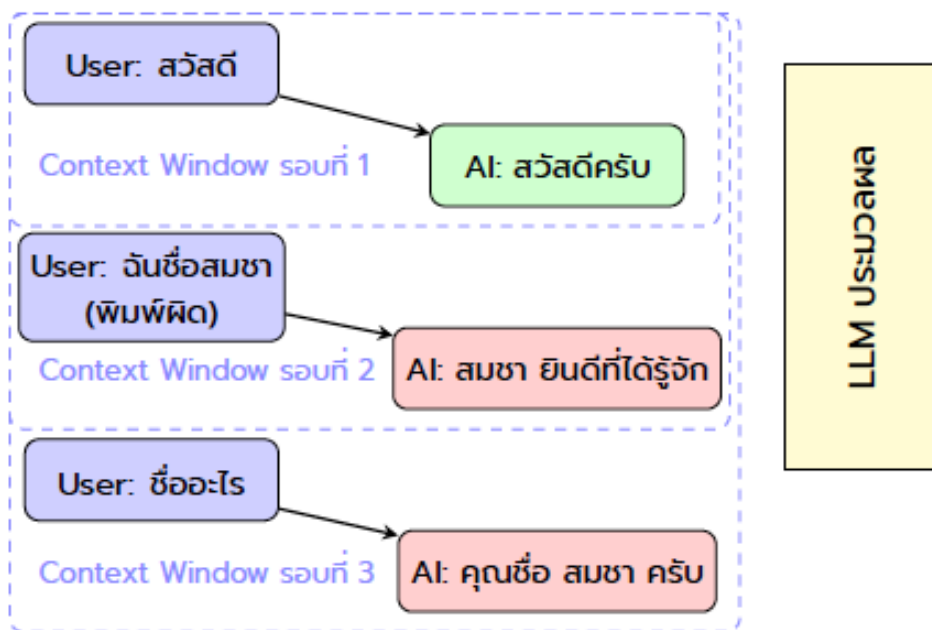
💡 Role Prompting ที่เราใส่ใน Prompt คือการ "จำลอง System" — นั่นคือเหตุผลที่มันส่งผลต่อโทนและมุมมองของคำตอบ



รูปที่ 3.3: โครงสร้างของ Chat History แสดง System Message ที่อยู่ตอนต้นและมีความสำคัญสูงสุด ตามด้วยการสนทนาย่หว่าง User และ AI ที่สะสมใน Context Window

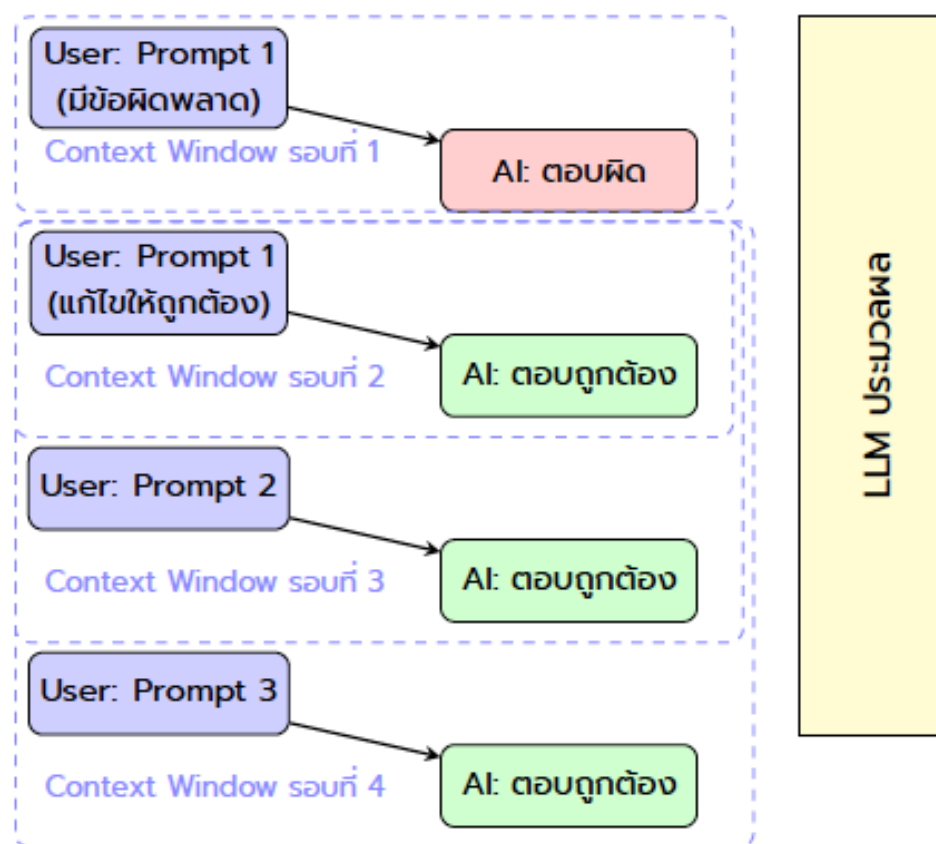


รูปที่ 3.4: กลไกการทำงานของ Chat History คือ ทุกข้อความถูกส่งเข้า Context Window ซ้ำ ๆ ในทุกรอบ ทำให้การสนทนาเป็นไปอย่างต่อเนื่อง



หมายเหตุ: ทุกรอบการสนทนา ระบบส่ง Chat History ทั้งหมดเข้า Context Window → ข้อผิดพลาดจะสะสมและแพร่กระจาย

รูปที่ 3.5: กลไกการทำงานของ Chat History คือ ทุกข้อความถูกส่งเข้า Context Window ซ้ำ ๆ ในทุกรอบ หาก Prompt แรกผิด ความผิดจะติดตามไปในทุกคำตอบถัดไป



รูปที่ 3.6: การแก้ไข Prompt ที่ผิดพลาดต้องทำที่ต้นทาง เพื่อป้องกันการสะสมข้อผิดพลาดในบริบทที่ส่งเข้าโมเดลในรอบถัดไป

Workshop 1 — Prompt ก่อน/หลัง

ลองทำ (10 นาที)

1. เลือกงานที่ทำบ่อยในชีวิตประจำวัน
2. เขียน Prompt แบบสั้นๆ ก่อน → ดูผลลัพธ์
3. ปรับ Prompt ด้วยสูตร **CO-STAR** → เปรียบเทียบ

คำถามชวนคิด: ผลลัพธ์ต่างกันอย่างไร?

ตัวอย่างเริ่มต้น:

- Prompt สั้น: "เขียนอีเมลขอเลื่อนนัดประชุม"
- Prompt CO-STAR: "[C] ฉันเป็นเจ้าหน้าที่ธุรการ [O] เขียนอีเมลขอเลื่อนนัดประชุมกับลูกค้า [S] ทางการ [T] สรุภาพ ขอโทษอย่างจริงใจ [A] ลูกค้าระดับผู้บริหาร [R] อีเมล 1 ย่อหน้า"

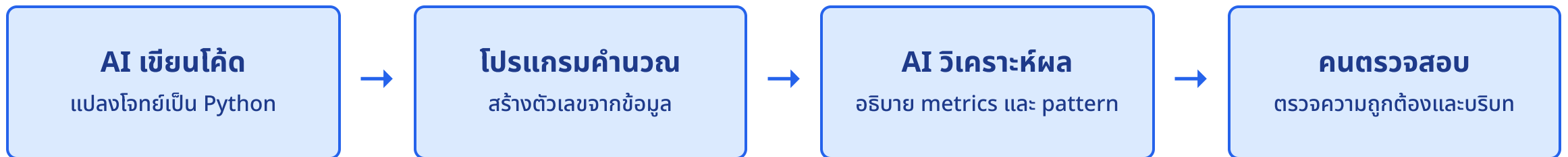
03

Data Analysis with Gemini

AI วิเคราะห์ข้อมูลอย่างไร?

AI ไม่ได้คำนวณตัวเลขด้วยตัวเอง

- LLM ช่วยแปลงคำถามให้เป็นขั้นตอนการวิเคราะห์
- AI เขียนโค้ดเพื่อการคำนวณ โดยที่นิยมจะเป็น Python สำหรับสร้างและประเมินโมเดล
- Python หรือเครื่องมือวิเคราะห์เป็นผู้คำนวณผลลัพธ์จริง
- AI ช่วยอธิบายผลลัพธ์ แต่ต้องมีมนุษย์ตรวจสอบ



เปรียบเทียบ Generative AI ในงานวิเคราะห์ข้อมูล

ไม่ได้แข่งกันที่ตัวโมเดลอย่างเดียว

เครื่องมือ	คู่หูหรือสภาพแวดล้อม	จุดเด่นในงานข้อมูล
Gemini	Google Colab	เขียนและรัน Python ใน Notebook เดียว
Claude	Artifacts	สร้าง prototype และเครื่องมือ interactive
ChatGPT	Data Analysis / Jupyter	อัปโหลดไฟล์แล้ววิเคราะห์ด้วย Python

ความสามารถจริงขึ้นกับรุ่นโมเดล แผนบริการ บัญชี และเครื่องมือที่เปิดใช้งาน

Gemini + Google Colab

ห้องทดลองข้อมูลที่มี AI อยู่ใน Notebook

- สร้างโค้ดจากคำสั่งภาษาธรรมชาติ
- อธิบายและแก้ไขโค้ด Python
- สร้างและรันโค้ดเพื่อวิเคราะห์ข้อมูล
- สร้างกราฟและช่วยสรุป insight
- ใช้ runtime, pandas และไลบรารี Data Science ใน Colab



จุดเด่น: ผู้เรียนเห็นชัดว่า AI ช่วยเขียนโค้ด แต่ Colab เป็นผู้คำนวณผลลัพธ์

Claude + Artifacts

พื้นที่สร้างเครื่องมือและ prototype

- แสดง code, document หรือ website ในพื้นที่แยกจากบทสนทนา
- สร้างเครื่องมือ interactive สำหรับทดลองกับข้อมูล
- เหมาะกับ HTML, CSS, JavaScript และ web prototype ตามรูปแบบที่รองรับ
- แก้ไขและพัฒนาผลงานต่อในพื้นที่เดียวกัน
- ใช้สร้าง dashboard หรือเครื่องมือสรุปผลเบื้องต้นได้

Artifacts เน้นด้านการสร้าง **เครื่องมือเพื่อใช้งาน** ไม่ใช่ Notebook สำหรับวิเคราะห์ข้อมูลโดยเฉพาะ

ตัวอย่างงาน

“สร้างหน้าเว็บสำหรับดูสถานะโครงการ แสดงงบประมาณ ความคืบหน้า และ KPI พร้อมตัวกรองตามโครงการ”

ChatGPT Data Analysis

นักวิเคราะห์ข้อมูลในแชต

- อัปโหลดไฟล์ CSV, Excel, PDF และไฟล์ข้อมูลอื่น
- สรุปแถว คอลัมน์ แนวโน้ม และ outlier
- สร้างตารางและกราฟจากข้อมูล
- เขียนและรัน Python ในสภาพแวดล้อมแบบ stateful Jupyter
- อธิบายโค้ด สมมติฐาน และผลลัพธ์ที่ได้

ควรขอให้ ChatGPT แสดง **โค้ดและผลลัพธ์** เพื่อให้ตรวจสอบวิธีคำนวณได้

ข้อจำกัดที่ควรรู้

- สภาพแวดล้อม Python ไม่ได้เรียกเว็บหรือ API ภายนอกได้โดยอิสระ
- หากข้อมูลอยู่บน URL อาจต้องดาวน์โหลดแล้วอัปโหลด หรือเชื่อมต่อแหล่งข้อมูลก่อน

ตัวอย่าง: Regression

คำถามเชิงข้อมูล

จำนวนผู้โดยสารสัมพันธ์กับค่าใช้จ่ายเชื้อเพลิงอย่างไร?

กำหนดตัวแปร

- **Target (y):** `Fuel Cost` ค่าใช้จ่ายเชื้อเพลิง (พันดอลลาร์)
- **Feature (X):** `Passenger` จำนวนผู้โดยสาร (คน)
- **เป้าหมาย:** สร้างสมการเพื่ออธิบายแนวโน้มและประมาณค่า Fuel Cost

Dataset: [Fuel Cost.xlsx](#)

1. ให้ AI เขียนโค้ด

```
import pandas as pd
from sklearn.linear_model import LinearRegression
url = "https://github.com/toche7/DataSets/raw/refs/heads/main/cost.xlsx"
df = pd.read_excel(url)
X = df[["Passenger"]]
y = df["Fuel Cost"]
model = LinearRegression().fit(X, y)
predictions = model.predict(X)
```

AI ช่วยสร้างโค้ด แต่ผู้ใช้ต้องตรวจ URL ชื่อคอลัมน์ หน่วยวัด และชนิดข้อมูลก่อนรัน

2. ให้โปรแกรมคำนวณ

```
from sklearn.metrics import mean_absolute_error, r2_score
print("Intercept:", model.intercept_)
print("Slope:", model.coef_[0])
print("R²:", r2_score(y, predictions))
print("MAE:", mean_absolute_error(y, predictions))
df["Prediction"] = predictions
df["Residual"] = y - predictions
```

2. ให้โปรแกรมคำนวณ (ต่อ)

ผลลัพธ์จาก Python

รายการ	ผลลัพธ์
Intercept	1.57
Slope ของ Passenger	+0.0407
R^2	0.899
MAE	0.1424 พันดอลลาร์

ตัวเลขเหล่านี้มาจากการรันโปรแกรม ไม่ใช่ตัวเลขที่ AI คาดเดา

3. ให้ AI วิเคราะห์ผล

สมการที่ได้จากข้อมูล

$$\widehat{Fuel\ Cost} = 1.57 + 0.0407 \times Passenger$$

ตัวอย่างคำอธิบาย

- เมื่อ Passenger เพิ่มขึ้น 1 คน ค่า Fuel Cost ตามสมการเพิ่มขึ้นประมาณ 0.04 พันดอลลาร์
- โมเดลอธิบายความแปรปรวนของ Fuel Cost ในข้อมูลชุดนี้ได้ประมาณ 86%
- ค่าคลาดเคลื่อนสัมบูรณ์เฉลี่ยประมาณ 0.17 พันดอลลาร์

4. มนุษย์ตรวจสอบก่อนใช้

- ตรวจสอบว่า **Passenger** และ **Fuel Cost** ใช้หน่วยวัดถูกต้อง
- ตรวจสอบว่าไฟล์ที่โหลดตรงกับ dataset ที่นำเสนอ
- ตรวจสอบ outlier และ residual ของแต่ละวัน
- ระงับการสรุปจากข้อมูลเพียง 7 แถว
- ห้ามสรุปว่า “ผู้โดยสารเพิ่มทำให้ค่าเชื้อเพลิงเพิ่ม” จาก regression เพียงอย่างเดียว

AI ช่วยเขียนและอธิบาย แต่ตัวเลขต้องมาจากโปรแกรม และข้อสรุปต้องผ่านการตรวจสอบของคน

ตัวอย่างที่ 2: รายงานความคืบหน้าโครงการ

คำถามสำหรับผู้บริหาร

จากหลายโครงการ โครงการใดบรรลุเป้าหมาย โครงการใดมีความเสี่ยง และควรติดตามเรื่องใด?

ข้อมูลที่ต้องใช้

ตารางที่ 1: แผนโครงการ

โครงการ	งบประมาณตามแผน	เป้าความคืบหน้า	KPI เป้าหมาย
บริการประชาชน	1,200,000	70%	1,000
บริการดิจิทัล	900,000	80%	5,000
อบรมบุคลากร	600,000	75%	800

ตารางที่ 2: ผลการดำเนินงาน

โครงการ	งบประมาณที่ใช้	ความคืบหน้าจริง	KPI ผลจริง
บริการประชาชน	780,000	62%	860
บริการดิจิทัล	720,000	85%	5,750
อบรมบุคลากร	510,000	58%	640

หน่วยงบประมาณ: บาท | KPI: คน

ตัวอย่างนี้เป็น **การสรุปและเปรียบเทียบข้อมูล** ไม่ใช่ Regression หรือการพยากรณ์

1. ให้ AI เขียนโค้ดสรุปโครงการ

```
import pandas as pd

projects = pd.DataFrame({
    "โครงการ": ["บริการประชาชน", "บริการดิจิทัล", "อบรมบุคลากร"],
    "งบประมาณตามแผน": [1_200_000, 900_000, 600_000],
    "เป้าความคืบหน้า": [70, 80, 75],
    "KPI เป้าหมาย": [1_000, 5_000, 800],
})
```

AI ช่วยเขียนโค้ด แต่ผู้ใช้ต้องตรวจสอบชื่อคอลัมน์ หน่วยวัด และนิยามของ KPI

2. ให้โปรแกรมคำนวณผลลัพธ์

ผลลัพธ์จาก Python

โครงการ	ใช้งบประมาณ	ต่างจากแผน	บรรลุ KPI	คงเหลือ
บริการประชาชน	65.0%	-8 จุด	86.0%	420,000
บริการดิจิทัล	80.0%	+5 จุด	115.0%	180,000
อบรมบุคลากร	85.0%	-17 จุด	80.0%	90,000

สถานะเบื้องต้น

- **บริการดิจิทัล: บรรลุเป้าหมาย** ผลจริงสูงกว่าแผน 5 จุด และ KPI อยู่ที่ 115%
- **บริการประชาชน: เผื่อระวัง** ใช้งบประมาณ 65% แต่ความคืบหน้าต่ำกว่าแผน 8 จุด
- **อบรมบุคลากร: ต้องเร่งรัด** ความคืบหน้าต่ำกว่าแผน 17 จุด และ KPI อยู่ที่ 80%

ผลลัพธ์นี้เกิดจากสูตรที่รันด้วยโปรแกรม ไม่ใช้การให้ AI เดาตัวเลข

3. ให้ AI ช่วยเขียนรายงาน

Prompt ที่ใช้

จากตารางผลการคำนวณของ 3 โครงการ ให้จัดกลุ่มสถานะเป็น บรรลุเป้าหมาย เฝ้าระวัง หรือ ต้องเร่งรัด โดยพิจารณาความต่างจากแผน การใช้งบประมาณ และการบรรลุ KPI จากนั้นเขียนรายงานสำหรับผู้บริหาร แยกเป็นภาพรวม โครงการที่ควรติดตาม และข้อเสนอแนะ ห้ามเพิ่มสาเหตุที่ไม่มีในข้อมูล

ตัวอย่างรายงานสั้น

จาก 3 โครงการ พบว่าโครงการบริการดิจิทัลบรรลุเป้าหมาย โดยมีความคืบหน้าสูงกว่าแผน 5 จุดและบรรลุ KPI 115% ส่วนโครงการบริการประชาชนควรเฝ้าระวัง เนื่องจากความคืบหน้าต่ำกว่าแผน 8 จุด ขณะที่ใช้งบประมาณไปแล้ว 65% โครงการอบรมบุคลากรควรเร่งรัดเป็นลำดับแรก เพราะความคืบหน้าต่ำกว่าแผน 17 จุดและบรรลุ KPI 80% ทั้งนี้ ข้อมูลชุดนี้บอกสถานะและช่องว่างของผลลัพธ์ แต่ยังไม่บอกสาเหตุของความล่าช้า

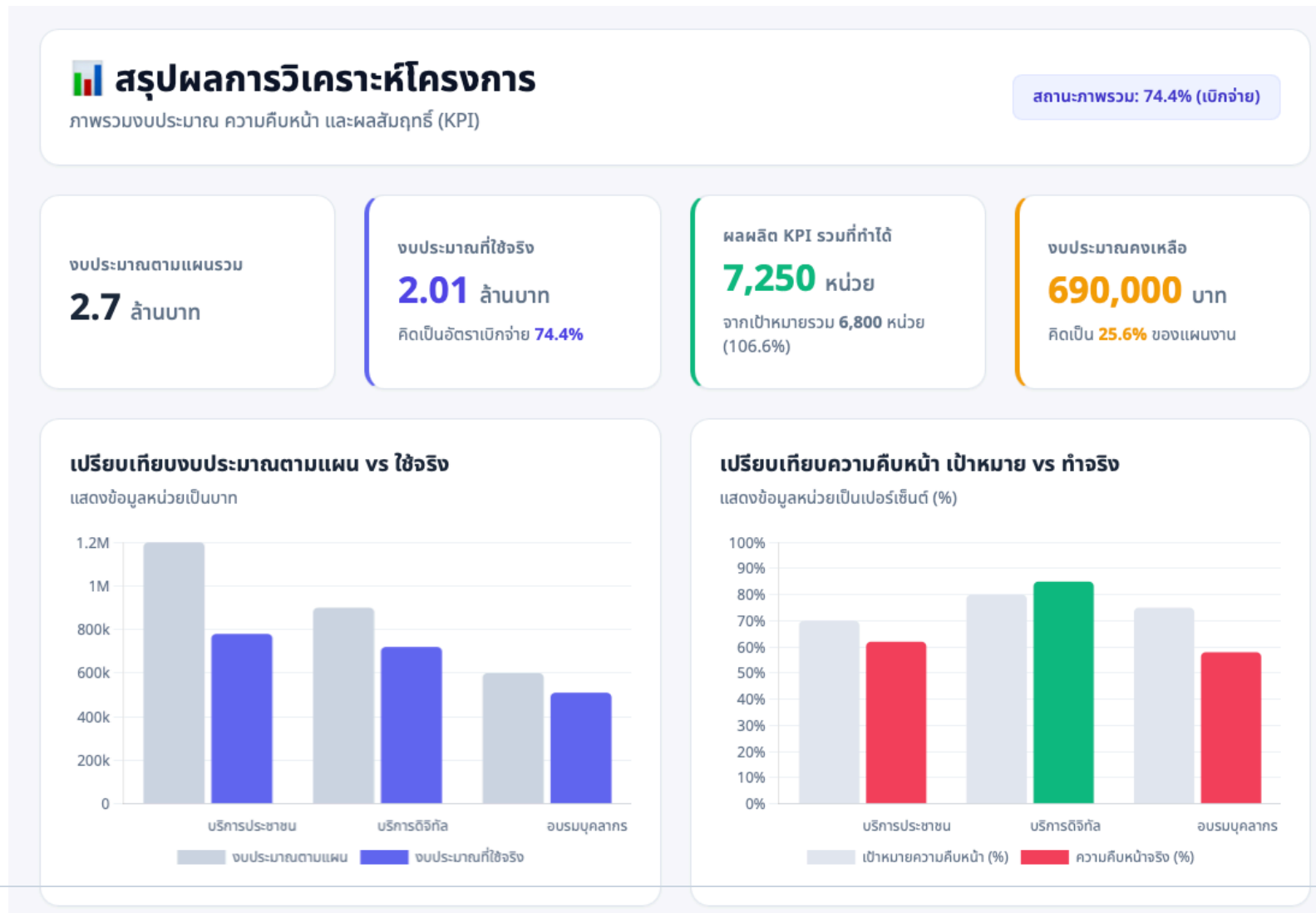
4. การสร้าง Dashboard ด้วย Canvas

ขั้นตอนการสร้าง

1. ให้ทำการวิเคราะห์ข้อมูลด้วยกระบวนการที่กล่าวมาและตรวจสอบความถูกต้องของผลลัพธ์
2. นำผลลัพธ์ที่ได้ไปสร้าง Dashboard ใน Canvas โดยใช้กราฟและตารางเพื่อแสดงสถานะของแต่ละโครงการ
3. ใน prompt input ให้กดเลือก option Canvas เพื่อให้ AI เปิด Canvas อัตโนมัติ
4. ใส่ prompt ดังนี้

"สร้าง Dashboard แสดงสถานะโครงการ 3 โครงการ: บริการประชาชน, บริการดิจิทัล, อบรมบุคลากร
โดยใช้กราฟแท่งและตาราง แสดงความคืบหน้า, การใช้งบประมาณ, และการบรรลุ KPI
พร้อมแยกสีตามสถานะ: บรรลุเป้าหมาย, เผื่อระวัง, ต้องเร่งรัด"

Example Dashboard Output



5. มนุษย์ตรวจสอบก่อนนำเสนอ

- ตรวจสอบข้อมูลของทุกโครงการใช้ช่วงเวลาและนิยามเดียวกัน
- ตรวจสอบงบประมาณที่ใช้ไปเป็นยอดเบิกจ่ายจริงหรือยอดผูกพัน
- ตรวจสอบความคืบหน้าวัดจากกิจกรรม น้ำหนักงาน หรือการใช้จ่าย
- ตรวจสอบ KPI และเกณฑ์การจัดสถานะ
- ไม่ให้ AI เพิ่มสาเหตุของความล่าช้าหากไม่มีข้อมูลรองรับ

งานรายงานโครงการ: โปรแกรมคำนวณสถานะ แล้ว AI ช่วยเรียบเรียงให้สื่อสารกับผู้บริหาร

Q&A

วิทยากร

ผศ.ดร.ทวิศักดิ์ สมานชื่น

Asst. Prof. Taweesak Samanchuen, Ph.D.

- รองผู้อำนวยการฝ่ายดิจิทัลเทคโนโลยี **MULKC**
- อาจารย์ประจำสาขา **ITM** คณะวิศวกรรมศาสตร์ มหาวิทยาลัยมหิดล
- หัวหน้าโครงการ **CBTU**

 [Profile](#)

 t.samanchuen@gmail.com

 081-441-4906

website: cbtumu.net | facebook: [cbtumu](https://www.facebook.com/cbtumu)

ขอบคุณครับ

ผศ.ดร.ทวิศักดิ์ สมนานชื่น